

Tarea 4 – Simulación estocástica

Lenin Pavón

2026-06-02

Para empezar importemos las siguientes bibliotecas:

- `tidyverse`
- `quantmod`. Para importación de datos de Yahoo Finance.
- `rugarch`. Para creación y ajuste de los modelos ARMA, GARCH.
- `fitdistrplus`. Para ajustes de distribuciones a los residuales.
- `MASS`. Para más pruebas de bondad de ajuste

Para que nuestro trabajo sea reproducible usaremos la semilla

```
set.seed(86)
```

Ahora cargaremos 2 activos,

```
ticker <- c("MSFT", "AAPL")
getSymbols(
  Symbols = ticker,
  src = "yahoo",
  from = "2025-01-01",
  auto.assign = TRUE
)
```

```
## [1] "MSFT" "AAPL"
```

Tenemos que ajustar los precios para ajustar por *stock split* (cuando una acción se divide en más) y pago de dividendos, para lo que usamos la función `Ad`. Posteriormente, los juntamos en un solo dataframe. Limitaremos a los

```
p <- as.numeric(MSFT$"MSFT.Adjusted")
q <- as.numeric(AAPL$"AAPL.Adjusted")
# Precios ajustados
n <- 200
precios <- data.frame(
  MSFT = p,
  AAPL = q
)
# Convertir a vector numérico
# colnames(precios) <- ticker
head(precios)
```

```
##      MSFT      AAPL
## 1 413.6727 242.3019
## 2 418.3867 241.8150
## 3 422.8339 243.4446
## 4 417.4182 240.6723
## 5 419.5826 241.1592
```

```
## 6 414.0383 235.3463
```

Ejercicio 1

Haga una prueba Ljung Box con lags 5, 10 y 15 individualmente para ver si los datos pueden considerarse como una muestra, posteriormente considere los rendimientos logarítmicos y adicionalmente a la prueba Ljung Box haga una prueba Akaike y grafique las funciones de autocorrelación para los lags antes mencionados.

Prueba Ljung-Box

Realizaremos esta prueba de autocorrelación para determinar si tenemos una muestra. Definimos los lags a los que realizaremos la prueba

```
lags <- c(5, 10, 15)
```

Realizamos la prueba

```
for (name in colnames(precios)){
  for (lag in lags) {
    res <- Box.test(
      precios[name],
      lag = lag,
      type = "Ljung-Box"
    )
    cat("asset: ", name, " | lag: ", lag, "\n")
    print(res)
  }
}
```

```
## asset:  MSFT  | lag:  5
##
##  Box-Ljung test
##
## data:  precios[name]
## X-squared = 1672.6, df = 5, p-value < 2.2e-16
##
## asset:  MSFT  | lag: 10
##
##  Box-Ljung test
##
## data:  precios[name]
## X-squared = 3198.5, df = 10, p-value < 2.2e-16
##
## asset:  MSFT  | lag: 15
##
##  Box-Ljung test
##
## data:  precios[name]
## X-squared = 4573.2, df = 15, p-value < 2.2e-16
##
## asset:  AAPL  | lag:  5
##
##  Box-Ljung test
##
```

```
## data: precios[name]
## X-squared = 1595.8, df = 5, p-value < 2.2e-16
##
## asset: AAPL | lag: 10
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 2944.5, df = 10, p-value < 2.2e-16
##
## asset: AAPL | lag: 15
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 4132.3, df = 15, p-value < 2.2e-16
```

Observemos que se rechaza la hipótesis de que los precios se comportan como una muestra aleatoria. De estas pruebas, podemos proceder con el resto del problema.

Rendimientos logarítmicos

Calculemos los rendimientos logarítmicos y eliminemos los la primera fila nula

```
ret_MSFT <- na.omit(diff(log(p)))
ret_AAPL <- na.omit(diff(log(q)))

ret <- data.frame(
  MSFT = ret_MSFT,
  AAPL = ret_AAPL
)
# colnames(ret) <- ticker
head(ret)
```

```
##           MSFT           AAPL
## 1  0.011331164 -0.002011531
## 2  0.010573316  0.006716337
## 3 -0.012890858 -0.011453115
## 4  0.005171661  0.002021009
## 5 -0.013301882 -0.024399105
## 6 -0.004209713 -0.010397916
```

Prueba Ljung-Box

Realizamos las mismas pruebas de Ljung-Box pero ahora sobre los rendimientos

```
for (name in colnames(ret)){
  for (lag in lags) {
    res <- Box.test(
      ret[name],
      lag = lag,
      type = "Ljung-Box"
    )
    cat("asset: ", name, " | lag: ", lag, "\n")
    print(res)
  }
}
```

```

}

## asset:  MSFT  | lag:  5
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 2.6526, df = 5, p-value = 0.7534
##
## asset:  MSFT  | lag: 10
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 4.9977, df = 10, p-value = 0.8913
##
## asset:  MSFT  | lag: 15
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 13.603, df = 15, p-value = 0.5558
##
## asset:  AAPL  | lag:  5
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 12.145, df = 5, p-value = 0.03285
##
## asset:  AAPL  | lag: 10
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 16.341, df = 10, p-value = 0.09027
##
## asset:  AAPL  | lag: 15
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 18.856, df = 15, p-value = 0.2203

```

De estos resultados vemos que sí se comporta como una muestra aleatoria.

Prueba Akaike

Definimos el arreglo dónde guardaremos los valores de las pruebas de Akaike.

```
aic_vals <- numeric(30)
```

Y realizamos la prueba para cada uno de estos valores

```

for (i in 1:15) {
  res <- arima(

```

```

    precios[name],
    order = c(i, 0, 0)
  )
  aic_vals[i] <- AIC(res)
}

```

```

## Warning in arima(precios[name], order = c(i, 0, 0)): possible convergence
## problem: optim gave code = 1

```

```

for (i in 16:30) {
  res <- arima(
    precios[name],
    order = c(i - 15, 0, 0)
  )
  aic_vals[i] <- AIC(res)
}

```

```

## Warning in arima(precios[name], order = c(i - 15, 0, 0)): possible convergence
## problem: optim gave code = 1

```

Organizando los datos de una forma tabular,

```

tabla_aic <- data.frame(
  idx = 1:15,
  MSFT = round(aic_vals[ 1:15], 4),
  AAPL = round(aic_vals[16:30], 4)
)
print(tabla_aic)

```

```

##      idx      MSFT      AAPL
## 1      1 2024.590 2024.590
## 2      2 2025.304 2025.304
## 3      3 2024.805 2024.805
## 4      4 2026.554 2026.554
## 5      5 2025.646 2025.646
## 6      6 2027.544 2027.544
## 7      7 2029.392 2029.392
## 8      8 2029.191 2029.191
## 9      9 2029.973 2029.973
## 10     10 2031.777 2031.777
## 11     11 2033.767 2033.767
## 12     12 2035.646 2035.646
## 13     13 2037.465 2037.465
## 14     14 2038.995 2038.995
## 15     15 2040.349 2040.349

```

Ahora veamos cual es el mejor orden para cada activo,

```

best_MSFT <- which.min(aic_vals[ 1:15])
best_AAPL <- which.min(aic_vals[16:30])

cat(
  "BestMejor modelo según AIC:\nMSFT: AR(",
  best_MSFT,
  ")\nAAPL: AR(",
  best_AAPL,
  ")\n",

```

```
sep = ""
)
```

```
## BestMejor modelo según AIC:
## MSFT: AR(1)
## AAPL: AR(1)
```

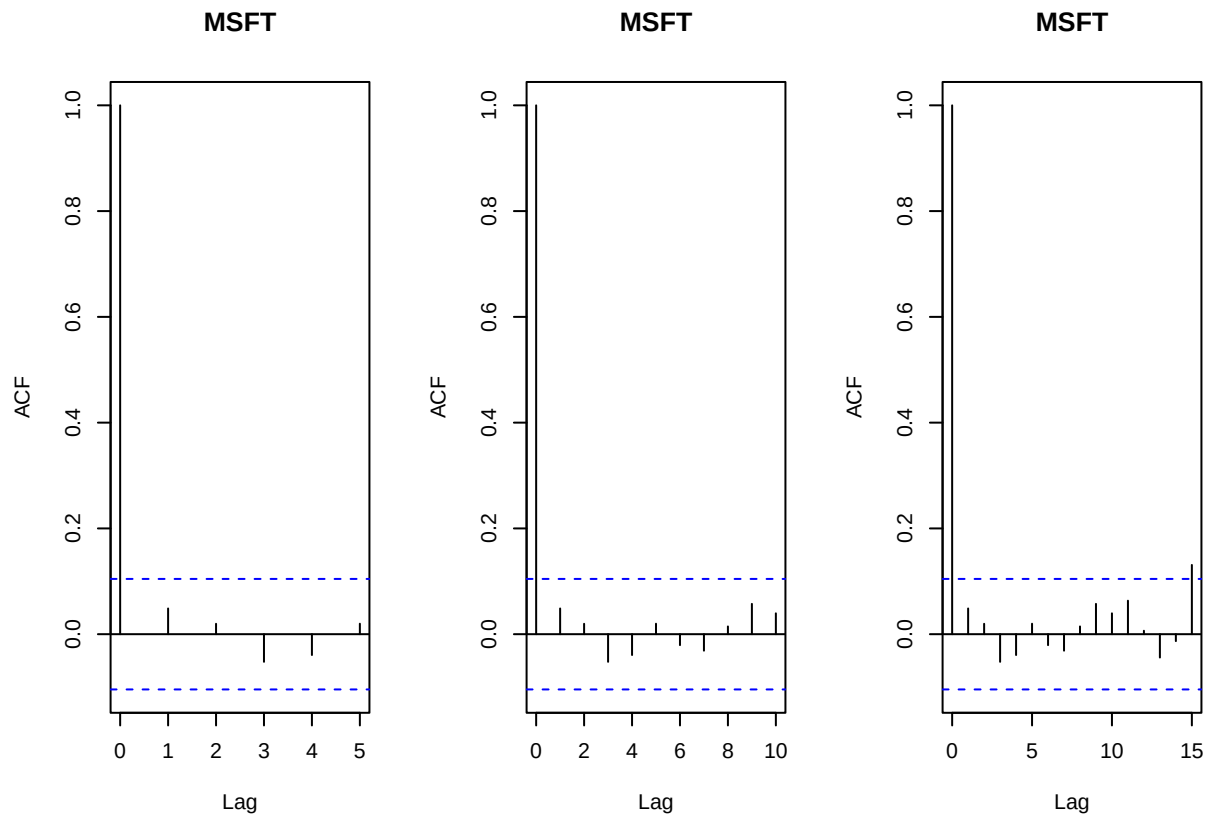
Gráficas de autocorrelación

Visualizando las gráficas de autocorrelación.

```
par(mfrow = c(1, 3))

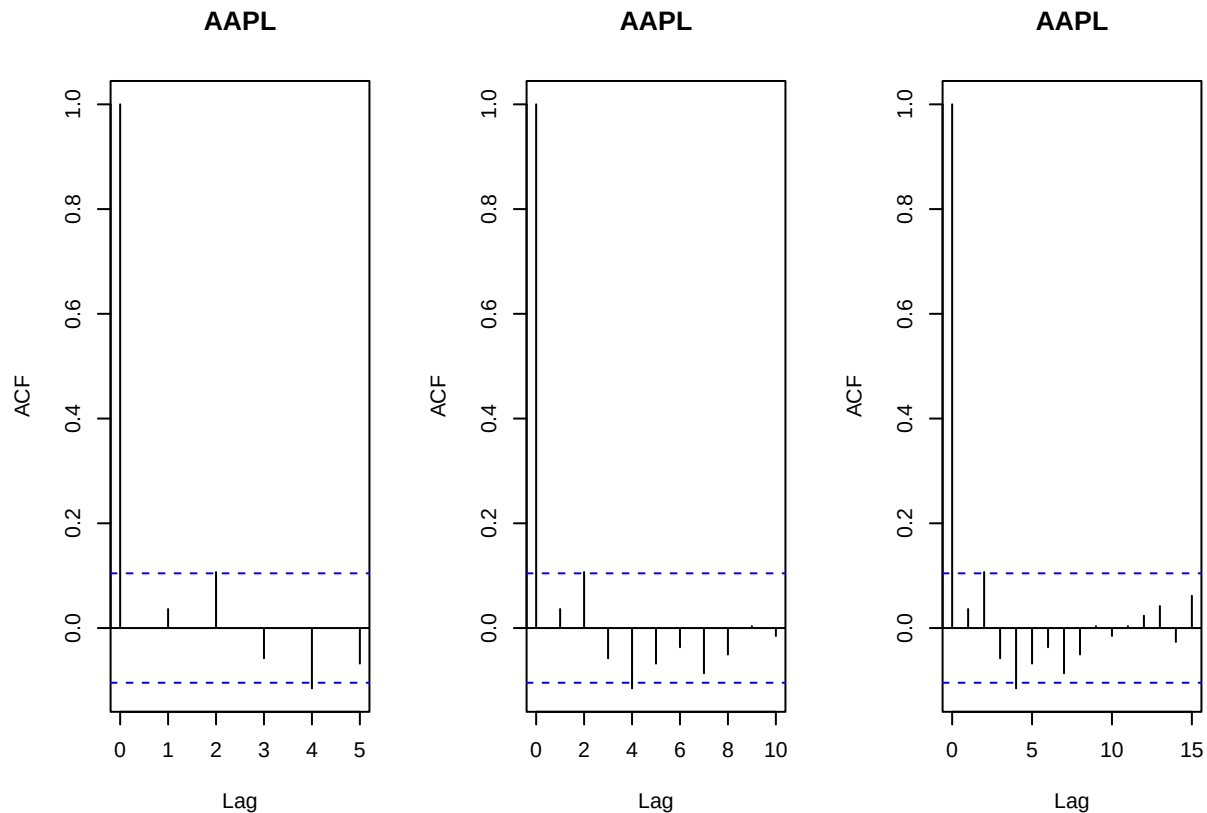
for (name in colnames(ret)){
  for (l in lags) {
    acf(
      ret[name],
      lag.max = l,
      main = cat(name, " - ACF - Lag ", l, sep = "")
    )
  }
}
```

```
## MSFT - ACF - Lag 5
## MSFT - ACF - Lag 10
```



```
## MSFT - ACF - Lag 15
## AAPL - ACF - Lag 5
```

```
## AAPL - ACF - Lag 10
```



```
## AAPL - ACF - Lag 15
```

```
par(mfrow = c(1, 1))
```

Ejercicio 2

Genere un ciclo para evaluar distintos parámetros de los modelos $\text{ARMA}(m, n) - \text{GARCH}(p, q)$. Considere $0 \leq m, n \leq 4$ y $0 \leq p, q \leq 2$. Presente la matriz con los resultados incluyendo las pruebas estadísticas a los residuales estandarizados de cada modelo así como los parámetros (ordénela de acuerdo a mayor p-value de la prueba Ljung-Box promedio). Diga qué modelo escogería para cada una de las dos series y por qué.

Generando el espacio de hiperparámetros sobre el que vamos a iterar cada serie

```
eval_model <- function(serie){
  resultados <- data.frame()
  for (cur_m in 0:4) {
    for (cur_n in 0:4) {
      for (cur_p in 0:2) {
        for (cur_q in 0:2) {
          # Saltamos iteración inválida
          if (cur_p == 0 && cur_q == 0) { next }
          # ID iteración
          cat(
            "ARMA(", cur_m, ",", cur_n,
            ") GARCH(", cur_p, ",", cur_q, ")\n"
          )
        }
      }
    }
  }
}
```

```

# Manejo de errores
tryCatch({
  # Modelo GARCH
  spec <- ugarchspec(
    variance.model = list(
      model = "sGARCH",
      garchOrder = c(cur_p, cur_q)
    ),
    mean.model = list(
      armaOrder = c(cur_m, cur_n),
      include.mean = TRUE
    ),
    distribution.model = "norm"
  )
  print("Modelo GARCH generado correctamente")
  # Ajuste
  ajuste <- ugarchfit(
    spec = spec,
    data = serie,
    solver = "hybrid"
  )
  print("Modelo GARCH ajustado correctamente")
  # Residuales
  resid_std <- residuals(
    ajuste,
    standardize = TRUE
  )
  print("Residuales estandarizados")
  # Prueba Ljung-Box sobre residuales
  lb_res <- Box.test(
    resid_std,
    lag = 15,
    type = "Ljung-Box"
  )
  print("Pruebas Ljung-Box")
  # Prueba Ljung-Box sobre residuales al cuadrado
  lb_sq <- Box.test(
    resid_std^2,
    lag = 15,
    type = "Ljung-Box"
  )
  print("Pruebas Ljung Box sobre el cuadrado")
  # Criterio de información para ev. de modelos
  ic <- infocriteria(ajuste)
  # Versión tabular de resultados
  resultados <- rbind(
    resultados,
    data.frame(
      AR    = cur_m,
      MA    = cur_n,
      GARCH = cur_p,
      ARCH  = cur_q,
      AIC   = ic[1],

```



```

        BIC = ic[2],
        LB_Resid = lb_res$p.value,
        LB_Resid2 = lb_sq$p.value,
        LB_Promedio =
            mean(
                c(
                    lb_res$p.value,
                    lb_sq$p.value
                )
            )
        )
    )
    print("Evaluado correctamente")
}, error = function(e) {print(e)})
}
}
}
}
resultados[
    order(
        resultados$LB_Promedio,
        decreasing = TRUE
    ),
]
}

```

Ahora generamos la estimación de los modelos a lo largo del espacio de las 224 combinaciones posibles para los activos MSFT y AAPL. Esto se realiza en unas celdas no incluida en el proceso final debido a que toma demasiado espacio.

Vamos a guardar los resultados en un csv

```

write.csv(
    tab_MSFT,
    "results/MSFT_res.csv",
    row.names = FALSE
)

write.csv(
    tab_AAPL,
    "results/AAPL_res.csv",
    row.names = FALSE
)

```

Personalmente, propongo seleccionar los modelos que permitan aumentar la parsimonia de los modelos. Sobre todo debido a la inmensidad del espacio de hiperparámetros.

Para MSFT, veamos que

```
head(tab_MSFT)
```

##	AR	MA	GARCH	ARCH	AIC	BIC	LB_Resid	LB_Resid2	LB_Promedio
## 107	2	3	1	0	15.266069	15.353879	0.9475852	1.0000000	0.9737926
## 151	3	3	2	1	-5.429380	-5.308641	0.8737206	0.9431520	0.9084363
## 190	4	3	2	1	-5.406956	-5.275241	0.8857464	0.9278052	0.9067758
## 51	1	1	1	0	76.772078	76.826959	0.8113864	1.0000000	0.9056932

```
## 11  0  1      1      0 79.392023 79.435928 0.8028040 1.0000000 0.9014020
## 43  1  0      1      0 76.779989 76.823894 0.8003066 1.0000000 0.9001533
```

Donde, debido a la similaridad en las métricas escogería

ARMA(2, 2) – GARCH(1, 1)

Para AAPL, veamos ahora que

```
head(tab_AAPL)
```

##	AR	MA	GARCH	ARCH		AIC	BIC	LB_Resid	LB_Resid2	LB_Promedio
## 26	0	3	1	0		6.158473	6.224331	0.9044588	0.9999979	0.9522284
## 140	3	2	1	2		-5.304053	-5.194291	0.9951826	0.6327694	0.8139760
## 143	3	2	2	2		-5.298371	-5.177633	0.9951789	0.6327487	0.8139638
## 190	4	3	2	2		-5.290142	-5.147451	0.9951759	0.6304243	0.8128001
## 179	4	2	1	2		-5.295959	-5.175221	0.9925675	0.6252907	0.8089291
## 182	4	2	2	2		-5.290277	-5.158563	0.9925617	0.6251366	0.8088491

Donde podríamos escoger a

ARMA(1, 0) – GARCH(1, 0)

Ejercicio 3

Considerando el modelo óptimo para cada serie, escoja una distribución para los residuales. Con dicha distribución obtenga la muestra de la cópula que se utilizará para estimar dicha cópula. Utilizando máxima verosimilitud estime los parámetros de las cópulas Gaussiana, t de Student, Gumbel, Clayton y Frank. Utilizando la distancia Cramer von Mises con la cópula empírica escoja la mejor.

Ajustes

Generemos los ajustes para cada activo,

```
spec_msft <- ugarchspec(
  variance.model = list(
    model = "sGARCH",
    garchOrder = c(2, 2)
  ),
  mean.model = list(
    armaOrder = c(1, 1),
    include.mean = TRUE
  ),
  distribution.model = "norm"
)
spec_aapl <- ugarchspec(
  variance.model = list(
    model = "sGARCH",
    garchOrder = c(1, 0)
  ),
  mean.model = list(
    armaOrder = c(1, 0),
    include.mean = TRUE
  ),
  distribution.model = "norm"
)
# Ajuste
fit_msft <- ugarchfit(
```

```

spec = spec_msft,
data = ret_MSFT,
solver = "hybrid"
)
fit_aapl <- ugarchfit(
spec = spec_aapl,
data = ret_AAPL,
solver = "hybrid"
)

```

Con los ajustes realizados estudiemos a los residuales estandarizados

```

z_msft <- as.numeric(residuals(fit_msft, standardize = TRUE))
z_aapl <- as.numeric(residuals(fit_aapl, standardize = TRUE))

```

Veamos qué distribución se ajusta mejor

```

ajustes_msft <- list(
normal = fitdist(z_msft, "norm"),
t = fitdistr(z_msft, "t"),
logistic = fitdist(z_msft, "logis")
)

```

```

## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced

## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced

```

```
sapply(ajustes_msft, AIC)
```

```

##      normal      t  logistic
## 1000.8796  940.0432  952.2198

```

De manera análoga para el activo AAPL

```

ajustes_aapl <- list(
normal = fitdist(z_aapl, "norm"),
t = fitdistr(z_aapl, "t"),
logistic = fitdist(z_aapl, "logis")
)

```

```

## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced

## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced

## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced

## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced

```

```
## Warning in log(s): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
```

```
sapply(ajustes_aapl, AIC)
```

```
##      normal      t logistic
## 696.5676 180.2851 314.4566
```

Vemos que la t de Student demuestra su superioridad, muy probablemente debido a las colas pesadas. Así, ajustaremos con una t y los juntaremos para la cópula

```
t_msft <- fitdistr(
  z_msft,
  densfun = "t"
)
```

```
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
```

```
t_aapl <- fitdistr(
  z_aapl,
  densfun = "t"
)
```

```
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in log(s): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
```

```
## Warning in log(s): NaNs produced
## Warning in dt((x - m)/s, df, log = TRUE): NaNs produced
## Warning in log(s): NaNs produced
df1 <- t_msft$estimate["df"]
df2 <- t_aapl$estimate["df"]

u <- pt(z_msft, df = df1)
v <- pt(z_aapl, df = df2)

datos_copula <- cbind(u, v)
```

Vamos a ajustar las cópulas

- Normal/Gaussiana,
- t de Student
- Gumbel
- Clayton
- Frank
- Empírica

Por máxima verosimilitud

```
# -----
# Normal
# -----
cop_gauss <- normalCopula(
  param = 0.5,
  dim = 2
)

fit_gauss <- fitCopula(
  cop_gauss,
  datos_copula,
  method = "ml"
)

# -----
# t
# -----
cop_t <- tCopula(
  param = 1,
  dim = 2,
  dispstr = "un",
  df = 2, df.fixed = TRUE
)

fit_t <- fitCopula(
  cop_t,
  datos_copula,
  method = "ml"
)

# -----
# Gumbel
# -----
cop_gumbel <- gumbelCopula(dim = 2)
```

```

fit_gumbel <- fitCopula(
  cop_gumbel,
  datos_copula,
  method = "ml"
)
# -----
# Clayton
# -----
cop_clayton <- claytonCopula(dim = 2)

fit_clayton <- fitCopula(
  cop_clayton,
  datos_copula,
  method = "ml"
)
# -----
# Frank
# -----
cop_frank <- frankCopula(dim = 2)

fit_frank <- fitCopula(
  cop_frank,
  datos_copula,
  method = "ml"
)
# -----
# Empírica
# -----
emp_cop <- empCopula(datos_copula)

```

¿Existe la mejor cópula?

Definimos la medición de distancia de Craver von Mises utilizando la función dada por `copula`.

```

calcular_cvm <- function(fit, datos) {
  cop <- fit@copula
  gofCopula(
    cop,
    x = datos,
    simulation = "mult"
  )$statistic
}

```

Para cada cópula (excepto la empírica) calculamos la distancia CvM

```

cvm_gauss <- calcular_cvm(
  fit_gauss,
  datos_copula
)

cvm_clayton <- calcular_cvm(
  fit_clayton,
  datos_copula
)

```

```

cvm_t <- calcular_cvm(
  fit_t,
  datos_copula
)

cvm_gumbel <- calcular_cvm(
  fit_gumbel,
  datos_copula
)

cvm_frank <- calcular_cvm(
  fit_frank,
  datos_copula
)

```

Veamos que como no se puede ajustar fácilmente para t , truncaremos el estimado de los grados de libertad a 2.

Mostrando los resultados de una forma tabular

```

resultados <- data.frame(
  Copula = c(
    "Gaussian",
    "Student-t",
    "Clayton",
    "Gumbel",
    "Frank"
  ),
  LogLik = c(
    logLik(fit_gauss),
    logLik(fit_t),
    logLik(fit_clayton),
    logLik(fit_gumbel),
    logLik(fit_frank)
  ),
  CvM = c(
    cvm_gauss,
    cvm_t,
    cvm_clayton,
    cvm_gumbel,
    cvm_frank
  )
)

resultados <- resultados[
  order(resultados$CvM),
]

print(resultados)

```

```

##      Copula   LogLik      CvM
## 1 Gaussian 1.2913297 0.01571307
## 3 Clayton 5.0851337 0.02086518
## 5 Frank   0.3223184 0.02115836
## 4 Gumbel  2.9490440 0.02253967

```

```
## 2 Student-t 9.8195904 0.03351674
```

Así la mejor cópula es la Gaussian. Se me hace interesante que la Gaussiana haya ajustado mejor. Habiendo dicho eso, debido a que estamos usando activos que están altamente correlacionados, no haya tantos eventos extremos haciendo que la Gaussiana haya respondido mejor. Sin embargo, la diferencia es menor a 10^{-2} por lo que en mayor o menor medida podríamos decir que sirven el mismo propósito. Por último, siento que una posible ventaja del uso de una cópula gaussiana es la parsimonia del modelo.

Ejercicio 4

Considerando a la cópula Gaussiana (sin importar cuál fue la mejor) y su parámetro ρ obtenido vía máxima verosimilitud y con la distribución de los residuales se construye la distribución conjunta de los residuales. Obtenga 200 simulaciones normales estándar bivariadas independientes. Con el método visto en clase genere 200 simulaciones de la cópula Gaussiana con correlación ρ .

Obtenemos la estimación mediante la cópula normal

```
rho_hat <- fit_gauss$estimate
```

Generamos n observaciones bivariadas independientes de la normal

```
n <- 200

z_ind <- matrix(
  rnorm(2 * n),
  ncol = 2
)

head(z_ind)
```

```
##           [,1]      [,2]
## [1,]  0.7019507  0.2583459
## [2,] -1.9561173  1.0004478
## [3,] -0.5959928  1.0208556
## [4,]  0.1119197 -0.6359005
## [5,]  0.5330768  0.1019308
## [6,]  1.8015164  0.8628476
```

Comprobamos

```
cor(z_ind)

##           [,1]      [,2]
## [1,]  1.00000000 -0.00281197
## [2,] -0.00281197  1.00000000
```

Para proceder definimos a A y obtenemos a L la matriz que se obtiene de la descomposición de Cholesky, es decir

$$A = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$$

que calculamos mediante

```
A <- matrix(
  c(
    1, rho_hat,
    rho_hat, 1
  )
```



```

),
nrow = 2,
byrow = TRUE
)
L <- chol(A)

```

Comprobamos que la correlación se acerque a 0.1640537

```

z_corr <- z_ind %*% L
cor(z_corr)

##           [,1]      [,2]
## [1,] 1.0000000 0.1624333
## [2,] 0.1624333 1.0000000

```

Simulando,

```

u <- pnorm(z_corr[, 1])
v <- pnorm(z_corr[, 2])

copula_sim <- data.frame(
  U = u,
  V = v
)

head(copula_sim)

```

```

##           U           V
## 1 0.75864504 0.6443100
## 2 0.02522567 0.7472896
## 3 0.27559003 0.8183908
## 4 0.54455647 0.2712874
## 5 0.70300978 0.5745628
## 6 0.96418924 0.8742477

```

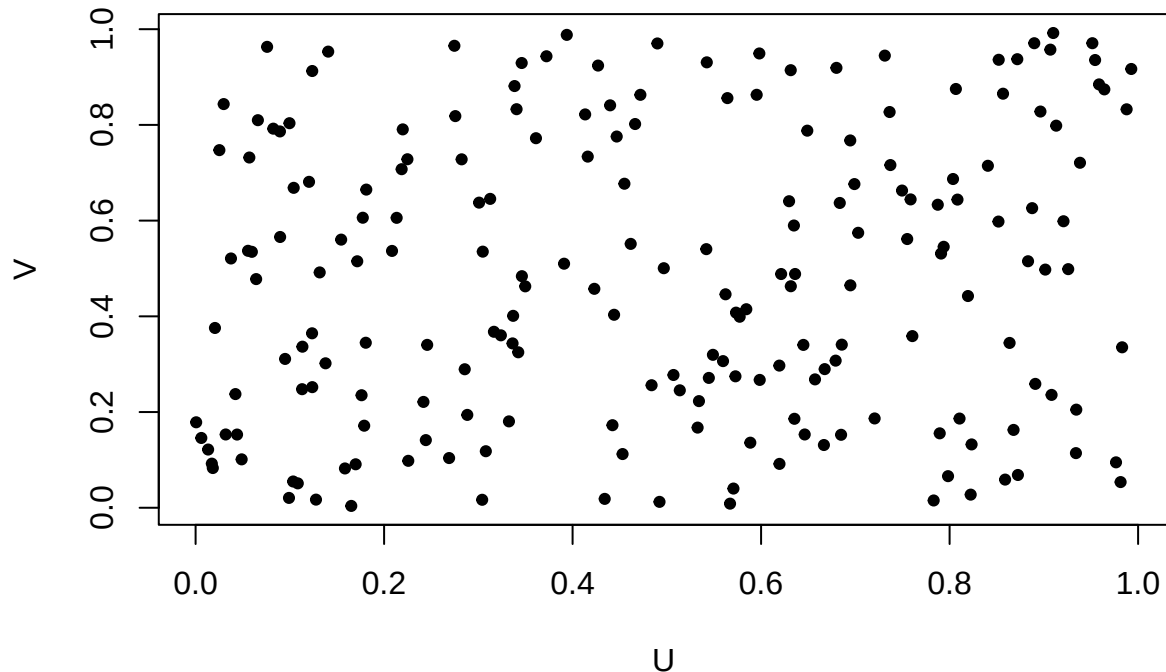
Graficamos

```

plot(
  copula_sim$U,
  copula_sim$V,
  pch = 19,
  cex = 0.7,
  main = paste(
    "Simulación Cópula Gaussiana",
    "rho =",
    round(rho_hat, 3)
  ),
  xlab = "U",
  ylab = "V"
)

```

Simulación Cópula Gaussiana $\rho = 0.164$



Adicional a la gráfica comparemos la transformación de bivariadas independientes a bivariada con correlación ρ

```
resultado <- data.frame(
  Z1 = z_ind[, 1],
  Z2 = z_ind[, 2],
  U = u,
  V = v
)

head(resultado, 10)
```

##	Z1	Z2	U	V
## 1	0.7019507	0.25834594	0.75864504	0.6443100
## 2	-1.9561173	1.00044783	0.02522567	0.7472896
## 3	-0.5959928	1.02085562	0.27559003	0.8183908
## 4	0.1119197	-0.63590045	0.54455647	0.2712874
## 5	0.5330768	0.10193075	0.70300978	0.5745628
## 6	1.8015164	0.86284759	0.96418924	0.8742477
## 7	0.6730807	0.31406779	0.74955202	0.6628427
## 8	1.0444165	0.07792999	0.85185363	0.5980158
## 9	1.0656359	0.94200965	0.85670587	0.8652182
## 10	-0.8119339	0.22831836	0.20841479	0.5366606

Ejercicio 5

Basado en dicha muestra genere 200 simulaciones los residuales, y 200 simulaciones de los precios originales. Considerando estas últimas calcule el VaR(95%) de un portafolio compuesto por 20 % de acciones del primer activo y 80 % del segundo

Pasaremos de las simulaciones obtenidas en el ejercicio anterior a residuales. De los residuales los convertiremos

a rendimientos. De ahí tomaremos el VaR del portafolio.

```
U <- copula_sim$U
V <- copula_sim$V
```

Recordando que estimamos los grados de libertad de los residuales utilizamos la inversa de la CDF

```
eps1 <- as.numeric(qt( U, df = df1 ))
eps2 <- as.numeric(qt( V, df = df2 ))
```

Ya que convertimos las simulaciones en residuales, guardémoslos en un dataframe

```
residuales_sim <- data.frame(
  eps1 = eps1,
  eps2 = eps2
)

head(residuales_sim)
```

```
##      eps1      eps2
## 1  0.7921152  0.4320986
## 2 -3.0582312  0.8203198
## 3 -0.6642550  1.1994242
## 4  0.1209872 -0.7405189
## 5  0.5903579  0.2157815
## 6  2.6520871  1.6556954
```

Obtenemos los parámetros de los ajustes

```
mu_msft      <- fitted(fit_msft)
sigma_msft    <- sigma(fit_msft)
mu_msft_last  <- as.numeric(tail(mu_msft, 1))
sigma_msft_last <- as.numeric(tail(sigma_msft, 1))

mu_aapl      <- fitted(fit_aapl)
sigma_aapl    <- sigma(fit_aapl)
mu_aapl_last  <- as.numeric(tail(mu_aapl, 1))
sigma_aapl_last <- as.numeric(tail(sigma_aapl, 1))
```

Obteniendo los rendimientos a partir de estos parámetros

```
r1_sim <- mu_msft_last + sigma_msft_last * eps1
r2_sim <- mu_aapl_last + sigma_aapl_last * eps2
```

Ahora obtenemos los últimos precios de los activos

```
p1 <- as.numeric(last(precios["MSFT"]))
p2 <- as.numeric(last(precios["AAPL"]))
```

De ahí realizamos las simulaciones

```
p1_sim <- p1 * exp(r1_sim)
p2_sim <- p2 * exp(r2_sim)
```

Construyamos ahora el portafolio, definamos su composición (pesos) y el valor inicial

```
w1 <- 0.20
w2 <- 0.80

v0 <- w1 * p1 + w2 * p2
```

De aquí obtenemos el valor total del portafolio tomando en cuenta la composición

```
v_sim <- w1 * p1_sim + w2 * p2_sim
```

Obtengamos los rendimientos del portafolio a partir del valor inicial

```
r_port <- (v_sim - v0) / v0
```

En conmemoración de los 18 años desde la publicación de *Loss*, llamemos a nuestra variable de pérdida `ctrlaltdel`

```
ctrlaltdel <- -r_port  
Loss <- ctrlaltdel
```

Calculemos el VaR.

```
var95 <- quantile( Loss, probs = 0.95 )  
cat("var95: ", var95, " | monetario: ", 1000 * var95)
```

```
## var95:  0.1992117  | monetario:  199.2117
```

Donde perderíamos alrededor de 19.9211668% del valor de nuestro portafolio en un día dado. Personalmente, sería demasiado riesgo para mí para una inversión a corto plazo. Sin embargo, a largo plazo, se amortizarían las pérdidas.

Agradecimientos

Agradezco la enorme paciencia que me tuvieron a lo largo del semestre. Aprendí mucho más de lo que esperaba (en gran medida en contra de mi voluntad) y espero que me sea de utilidad a lo largo de mi vida profesional.